

HEXA

Governance

The control and trust layer. Protects identity, permissions, policy, budgets, approvals and audit evidence so every mission stays inside the user's authority.

Made for: product, engineering, operations and business teams.
Reading time: about 12 minutes.
Truth standard: describes the repository as implemented on 1 August 2026.

HEXA

What HEXA is here to do

A role, not an all-powerful chatbot

In one sentence

Protects identity, permissions, policy, budgets, approvals and audit evidence so every mission stays inside the user's authority.

3

product areas grouped here

2

registered callable tools

4

allowed capability prefixes

0

agents it may delegate to

Receives

Verified user and tenant identity · Requested capability and permission ·
Proposed action plan · Approval record and audit context

Produces

Allow or refuse result · Preflight and post-verification evidence ·
Approval requirement · Tamper-evident audit trail

Allowed capability families

agent.*

workflow.*

governance.*

general.*

The key distinction

The product-area grouping below shows where HEXA contributes across XELOR. The callable-tool table later shows the smaller, exact surface Agent OS can invoke today. Product ownership does not automatically create runtime authority.

The business areas under HEXA

Business ownership is broader than today's Agent OS tools

Organisation

Company and organisation setup used to give every record the right business context.

Administration

Users, roles, permissions, separation of duties, security and compliance controls.

Integration

Reliable connections, retries, idempotency, circuit breakers, webhooks and adapter health.

How to read this grouping

HEXA is the named specialist when a mission needs facts or analysis from these areas. Each module still owns its own rules, records and write paths. The agent coordinates through registered services; it does not become the database owner.

What remains true underneath

- Each module keeps its own business rules and permissions.
- Cross-module work passes through narrow ports or registered capabilities.
- Tenant isolation, audit and workflow apply regardless of which agent is speaking.
- A readable agent answer never replaces the source record.

How HEXA participates in a mission

The mission engine controls order, parallel work and recovery



1 · Scope

ONYX turns the goal into a versioned graph run with a fixed tenant, user, limits and expected outputs.

2 · Authorise

The capability registry checks that HEXA is allowed to call the named tool; RBAC repeats the user permission check.

3 · Execute safely

The registered domain service performs the operation. The agent never receives a general SQL doorway.

4 · Preserve evidence

Inputs, outputs, node events and checkpoints are stored so the mission can be explained or recovered.

Delegation

HEXA is a specialist and does not delegate to other agents in the current registry.

What HEXA can call today

Every row is explicitly registered and permission-checked

Capability key	Mode	Result	User permission	Approval?
general.companies.read	Read	Company master	general.company.read	No
agent.action.dispatch	Execute	Governed work item	agentos.run.operate	Yes

What “dispatch” means

If listed, `agent.action.dispatch` appends a governed work item for a named specialist. It does not directly change a sales order, stock balance, production order, journal, payment or customer message.

Read / analyse / simulate

Returns evidence or a reproducible view. No business record is changed.

Draft

Creates reviewable proposed content. A draft is never labelled as an approved or completed business outcome.

Execute

Effectful at the Agent OS level and therefore requires an approved ancestor node.

Permission

The agent's allow-list and the signed-in user's RBAC must both allow the call.

Who HEXA works with

Specialists exchange evidence through ONYX, not hidden side conversations

ONYX	Returns a clear control decision and the evidence behind it.
Every specialist	Applies the same tenant, RBAC and approval boundary to its calls.
Human approver	Preserves who approved what, when, and for which exact plan.

A complete controlled mission



Why this structure matters

Each agent stays accountable for its own facts. ONYX combines them, HEXA checks control evidence, and a person controls the decision boundary. This prevents one fluent answer from quietly becoming authority over the whole company.

Is this controlled action genuinely authorised?

A realistic presentation path from question to controlled outcome

1

HEXA checks the signed-in user, tenant, requested permission, graph version and action plan before approval is requested.

2

The run pauses. A recorded human decision applies only to the exact waiting approval node and its mission context.

3

After dispatch, HEXA checks the stored work items and audit events; ONYX only receives a verified status, not an unsupported success claim.

What the presenter should say

“HEXA contributes verified domain evidence, with source references and limitations. It does not claim that a proposed or dispatched action is complete until the controlled system records and verifies the outcome.”

Useful questions to ask during the demo

- Which source records support HEXA's answer?
- What permission was checked?
- Is this a fact, a simulation, a draft or a completed result?
- Would this step wait for human approval?
- What happens if a node times out or the service restarts?

How HEXA stays trustworthy

Controls are enforced in the runtime and modules, not left to prompt wording

Control 1

Tenant is taken from verified identity, never a casual request header

Control 2

Agent calls repeat the user's RBAC check

Control 3

Side effects require approval in the graph ancestry

Control 4

Audit and dispatch records are append-only

Identity boundary

Verified user + tenant

Authority boundary

Agent allow-list + user RBAC

Action boundary

Approval ancestry for side effects

Data boundary

Domain service + tenant-fenced database

Evidence boundary

Append-only run, event and audit records

What is live—and what is not

This page prevents the demo from overclaiming

Live in the repository

- RBAC and tenant fencing
- Approval preflight and verification
- Budgets, opt-out and kill switch
- Audit evidence
- Resilient integration patterns

Deliberate current limits

- External connectors are not active in the demo runtime
- Governance can allow or block work; it does not replace a human owner
- Policy quality still depends on correct role and workflow configuration

Runtime disclosure

Orchestration, ERP reads, approval gates and governed action dispatch are live. Language reasoning is deterministic; no external model API or connector is active.

Primary implementation references

`apps/api/src/agent-os/capability-registry.service.ts`

`apps/api/src/agent-os/agent-action.service.ts`

`apps/api/src/agent-os/agent-authorization.service.ts`

This guide is a plain-language operating map, not an API contract. The code, migrations and automated tests remain the binding technical source.

HEXA in one page

Use this page when explaining the agent to a new team member

Job

Protects identity, permissions, policy, budgets, approvals and audit evidence so every mission stays inside the user's authority.

Owns

Organisation, Administration, Integration

Receives

Verified user and tenant identity · Requested capability and permission · Proposed action plan · Approval record and audit context

Returns

Allow or refuse result · Preflight and post-verification evidence · Approval requirement · Tamper-evident audit trail

Can call

general.companies.read, agent.action.dispatch

Cannot do

External connectors are not active in the demo runtime · Governance can allow or block work; it does not replace a human owner · Policy quality still depends on correct role and workflow configuration

Plain-language glossary

Agent

A named specialist role with an allow-listed set of tools—not a free-roaming process.

Capability

One registered operation with an input, output, owner, mode and required permission.

Graph

A versioned mission recipe showing order, parallel steps, joins and approvals.

Checkpoint

A stored progress snapshot used for explanation and safe recovery.

Governed work item

An append-only assignment created after approval; it is not the final business mutation.